Project Planning: `DataMetaMap` Library

Project Name: `DataMetaMap` — A Library for Dataset and Task Embeddings

Project Scope: Implementation of four task/dataset embedding algorithms:

1. **Maximum Mean Discrepancy (MMD)** — baseline, kernel-based approach
2. **Task2Vec** — Fisher Information Matrix diagonal from probe network
3. **Dataset2Vec** — DeepSets-based approach for tabular data
4. **Wasserstein Task Embedding** — optimal transport-based embedding

Features:

- Unified Python API for all methods
- Modular architectures
- Integration examples: transfer learning, dataset similarity search

Project Stack & Architecture

Core Libraries:

- **PyTorch** — deep learning backend для Task2Vec, Dataset2Vec
- **POT (Python Optimal Transport)** — для Wasserstein Embedding
- **scikit-learn** — kernels, clustering, metrics
- **NumPy/Pandas** — базовые операции с данными
- **matplotlib/seaborn** — визуализация

Architecture Pattern — Base Classes:

`BaseEmbedder` — unified interface for all embedding methods:

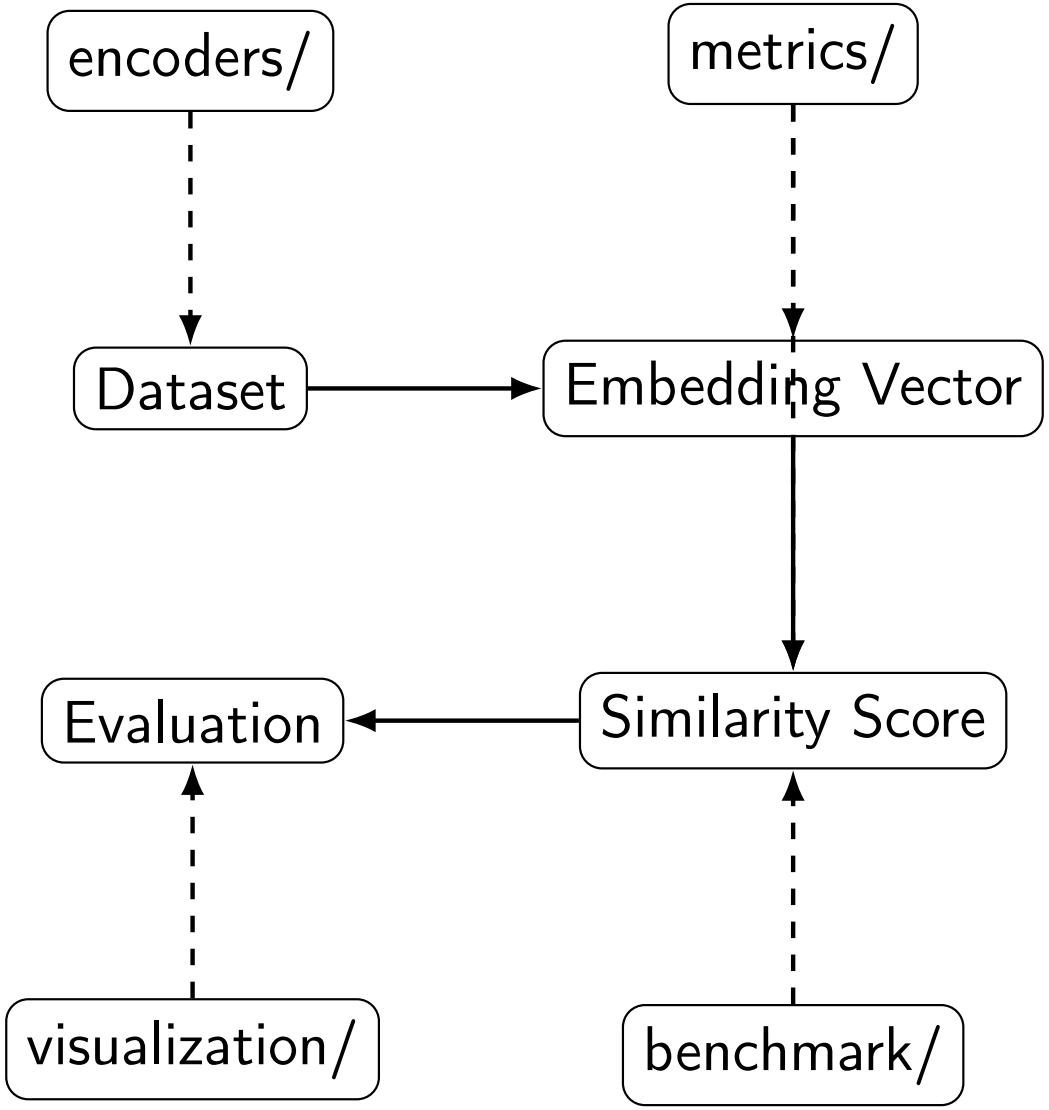
`Task2VecEmbedder` | `Dataset2VecEmbedder` | `WassersteinEmbedder`

`BaseMetric` — similarity and distance metrics:

`BaseEncoder` — feature extraction backends:

`IdentityEncoder` | `PretrainedEncoder` | `TrainableEncoder`

Project Stack & Architecture



Maximum Mean Discrepancy (MMD)

Key Insight: Embed probability distributions into a Reproducing Kernel Hilbert Space (RKHS) and measure the distance between their mean embeddings.

How it works:

1. Map samples from two datasets $X \sim P$ and $Y \sim Q$ into RKHS using feature map $\phi(\cdot)$
2. Compute mean embeddings: $\mu_P = \mathbb{E}[\phi(X)]$, $\mu_Q = \mathbb{E}[\phi(Y)]$
3. $\text{MMD}^2(P, Q) = \|\mu_P - \mu_Q\|_{\mathcal{H}}^2 = \mathbb{E}[k(X, X')] + \mathbb{E}[k(Y, Y')] - 2\mathbb{E}[k(X, Y)]$

where $k(\cdot, \cdot)$ is a characteristic kernel (e.g., Gaussian RBF)

Properties:

- Non-parametric: no density estimation required
- Characteristic kernel ensures $\text{MMD} = 0 \Leftrightarrow P = Q$
- Works in high dimensions without curse of dimensionality
- $O(n^2)$ computation, $O(n)$ approximations available

Как сравнивать датасеты

Домен: Любые векторизуемые данные (табличные, изображения, тексты). Требуется выбор ядра $k(\cdot, \cdot)$, соответствующего типу данных.

Структура эмбеддинга датасета:

- Представляем датасет как эмпирическое распределение $\hat{\mu} = \frac{1}{N} \sum_{i=1}^{N} \phi(x_i)$
- Для сравнения двух датасетов используем MMD как метрику близости

Варианты реализации:

- **Базовый MMD:** непосредственное вычисление по сэмплам
- **MMD с предобученным энкодером:** $\phi(x) = f_{\text{pretrained}}(x)$
- **Deep Kernel MMD:** обучаемое ядро $k_\theta(x, y) = \langle \phi_\theta(x), \phi_\theta(y) \rangle$

Применения:

- MMD^2 как скор различия
- k-NN на эмбеддингах для поиска схожих датасетов
- Кластеризация/визуализация (MDS/t-SNE)

TASK2VEC: Method Overview

Key Insight: Use the diagonal of the **Fisher Information Matrix (FIM)** of a pre-trained *probe network* as a task embedding.

How it works:

1. Take a probe network (e.g., ResNet-34 pre-trained on ImageNet).
2. For a new task (dataset with labels $\mathcal{D} = \{(x_i, y_i)\}$), re-train **only the final classifier layer**.
3. Compute the diagonal of the FIM for the feature extractor's parameters.
4. The resulting fixed-size vector is the **TASK2VEC** embedding.

Properties:

- Fixed dimension (number of filters), independent of the number of classes.
- Norm $\|F\|$ correlates with task difficulty.
- Cosine distance between embeddings reflects semantic similarity.

Similarity Metrics for Tasks

1. Symmetric Metric (Semantic Similarity)

Measures how alike two tasks are in terms of visual features needed.

$$d_{\text{sym}}(F_a, F_b) = d_{\cos}\left(\frac{F_a}{F_a + F_b}, \frac{F_b}{F_a + F_b}\right)$$

- F_a, F_b: task embeddings (diagonals of FIM).
- Division is element-wise; $d_{\cos}$ is cosine distance.
- **Use case:** Clustering tasks, analyzing taxonomic similarity.

2. Asymmetric Metric (Transferability)

Predicts how well a model trained on task a will perform on task b.

$$d_{\text{asym}}(t_a \to t_b) = d_{\text{sym}}(t_a, t_b) - \alpha\, d_{\text{sym}}(t_a, t_0)$$

- t_0: trivial embedding (isotropic prior, low complexity).
- Penalizes overly complex source tasks; α is a hyperparameter.
- **Use case:** Selecting the best pre-trained model for a new task.

Wasserstein Task Embedding

Домен: Любые векторизуемые данные с размеченными классами(label) и с некоторыми условностями. Объекты сравниваемых датасетов должны иметь векторное представление одной размерности. А сами датасеты должны представлять одинаковое количество классов - иначе танцы с бубном.

Структура:

- Группируем данные в датасете по классовым label-ам. Получаем $\mathcal{X} = \{x_n\}_{n=1}^{N}$ для каждого класса
- Это $\mathcal{X}$ мы с помощью MDS сжимаем до размерности N на l
- Каждый объект конкатенируется с его MDS по label-у: $x_i := [x_i, MDS(y_i)]$ из $dim(x_i) + l$ элементов

Как сравнивать датасеты

1. После конкатенации признаков и MDS-эмбеддингов меток, каждый датасет τ_i превращается в набор точек τ_i' в пространстве $\mathbb{R}^{d+l}$.
2. Мы рассматриваем датасет как эмпирическое распределение вероятностей μ_i (равномерное распределение на точках τ_i').
3. Используем **Wasserstein Embedding (WE)**:
   - Фиксируем эталонное распределение μ_0 (случайный набор точек-шаблонов).
   - Для каждого датасета μ_i находим оптимальную транспортную карту T_i, переводящую μ_0 в μ_i.
   - Эмбеддингом датасета будет вектор $\Phi(\mu_i) = (T_i - id)/\sqrt{N_0}$.
4. **Результат:** Евклидово расстояние между векторами $\Phi(\mu_i)$ и $\Phi(\mu_j)$ приближает оптимальное транспортное расстояние (OTDD) между исходными датасетами.

Dataset2Vec

Домен:

Табличные данные.

Структура:

$$\hat{\phi}(D) = h\left(\frac{1}{MT}\sum_{m=1}^{M}\sum_{t=1}^{T} g\left(\frac{1}{N}\sum_{n=1}^{N} f(x_{n,m}, y_{n,t})\right)\right),$$

где f, g, h — обучаемые блоки (DeepSets-подход). Инвариантность к перестановкам строк/столбцов.

Как учится:

- Генерируем пары батчей: *позитивные* — два подбатча из одного датасета, *негативные* — из разных датасетов.
- Сравнение эмбеддингов через ядро:

$$\hat{i}(x, x') = \exp\big(-\gamma\,\|\hat{\phi}(x) - \hat{\phi}(x')\|\big).$$

- Минимизируем бинарный лог-loss по парам:

$$\min_{\phi} -\frac{1}{|P|}\sum_{(x,x')\in P}\log\hat{i}(x, x') - \frac{1}{|W|}\sum_{(x,x')\in W}\log(1 - \hat{i}(x, x')).$$

Как сравнивать датасеты

Как сравнивать:

- **k-NN** по $\hat{\phi}(D)$ — найти похожие датасеты.
- **Ядро/скор** $\hat{i}(x, x')$ — мера близости.
- **Группировка/визуализация** (MDS / t-SNE) для анализа кластеров датасетов.
- **HPO warm-start**: взять лучшие конфигурации гиперпараметров с близких датасетов, оценить ускорение через ADTM(это метрика, которая показывает, насколько быстро алгоритм приближается к лучшему возможному результату.).
- **Pairwise classification accuracy**: взять отсечку для ядровой меры близости и просчитывать accuracy.